A cup of MOCCa

W. Ryssens*
Institut d'Astronomie et d'Astrophysique, Université Libre de Bruxelles, Campus de la Plaine CP 226, 1050 Brussels, Belgium

M. Bender
IPNL, Université de Lyon, Université Lyon 1, CNRS/IN2P3, F-69622 Villeurbanne, France

P.-H. Heenen
PNTPM, CP229, Université Libre de Bruxelles, B-1050 Bruxelles, Belgium
(Dated: Januart 2022)

Many thanks to a lot of collaborators who have used the code and given feedback, but in particular to G. Scamps, who endured many segfaults and non-converging calculations for extreme large scale testing of the code.

I. INTRODUCTION: TANTALUS AND HEPHAESTOS

Tantalus in its current incarnation is extremely versatile: the code (i) offers a great many options for the user to impose (or not) different selfconsistent symmetries and (ii) a (in principle) very large variety of different forms of the Skyrme EDF. Offering a combination of all of these possibilities in one executable (or equivalently, one set of source code files) is **very** complicated, as I found out with MOCCav1. The other extreme, writing many separate codes becomes a maintenance nightmare and bugfixing party, as I found out during my early days in the PNTPM at ULB. This is why the architecture of Tantalus, MOCCav2, is very different: as much as possible of what I call "complicated choices" (comprising, but not limited too, the specifics of the form of the EDF and symmetry options) are decided **compile-time**. There exists a meta-code in my private repository, called Hephaestos, that writes appropriate Tantalus code based on a particular instance of "complicated choices". In pseudo-code:

```
python Hephaestos < Hephaestos.in > [Tantalus code]
```

Different inputs to Hephaestos produce different source code for Tantalus, which can then be compiled into a separate executable. This manual does not document any aspect of Hephaestos, instead focussing on the commonalities and differences of the multiple resulting sets of Tantalus source code.

II. COMPILATION AND ALL DIFFERENT SOURCE CODE SETS

There are currently four different code-sets in the repository. These are:

- **src/** : Default option, maximally symmetric executable, named Tantalus.NLO.func.exe by default.
- **src_T/**: Time-reversal breaking, but spatially maximally symmetric. Tantalus.NLO.func.T.exe by default.
- **src_P/** : Parity breaking, time-reversal symmetric. Tantalus.NLO.func.P.exe by default.
- **src_TP/** : Parity and time-reversal breaking, named Tantalus.NLO.func.TP.exe by default.

More information is summarized in Table I. The makefile provided with the repository allows for easy compilation of all of these. A simple make command will compile **all** of these executables, but individual ones can be compiled with the commands listed in the first column of Table. I. The compiled executables can all be found in the exec/ folder. Be default, the code is compiled with the gfortran compiler. While this is very portable and compiles comparatively quickly, this is a **suboptimal** choice for production use: at least on the machines available in Brussels, compiling with **ifort** results in executables that are 20% faster or more. One can specify the compiler to the makefile, by typing

* wryssens@ulb.be

```
make CXX=ifort
```

If you feel confident, you can add extra flags to the compilation process by using the CXXFLAGS flag, but I suggest reading the Makefile first in that case.
Since the different executables and compileoptions can quickly give rise to confusion. For this reason, the Makefile will include some information on the actual compilation options in the header of the code, such as for example:

```
 ------------------------------------------------------------
|                                                            |
| MOCCa v2.0 =                                               |
|                                                            |
|  #######   ##   #    # #####   ##   #      #    #  ####   |
|     #     #  #  ##   #   #    #  #  #      #    # #       |
|     #    #    # # #  #   #   #    # #      #    #  ####   |
|     #    ###### #  # #   #   ###### #      #    #      #  |
|     #    #    # #   ##   #   #    # #      #    # #    #  |
|     #    #    # #    #   #   #    # ######  ####   ####   |
|                                                            |
|  Copyright  P.-H. Heenen, M. Bender & W. Ryssens           |
|                                                            |
|------------------------------------------------------------|
| Runtype = Single-mode                                      | => Deprecated information
|-------------- Version Information ------------------------|
|                                                            |
| "commit bb9a0523b8b85c571e30ea524aa3bac47901db42"          | => Precise git commit reference
| "Author: Wouter@Orome <wouter.ryssens@gmail.com>"          |
| "Date:   Tue Jan 11 20:47:32 2022 +0100"                   |
|                                                            |
|-------------- Symmetry Information ----------------------|
|                                                            | > Symmetries imposed on the nucleus
| S.p. generators        = Rz,T,STy                          |    can be combination of Rz, P, T, STy
| Axis reduction  X Y Z  =  1 1 0                            | > Axis reduction: which axes are fully (0)
| SYM_CODE               = 0 0 001 000 10 000 010 110        |    represented or not (1).
| TRANS_CODE             = 0 1 001 000 10 000 010 111        | > Codes referring to input and output of wfs
|-------------- Environment Information -------------------| > Not used yet
|-------------- Compilation Information -------------------|
| Compiled with:                                             |
| "GNU Fortran (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0"         | > output of "$CXX --version"
| Compilation flags reported:                                |
| "-O3 -Jmod -Wall -Wno-uninitialized  "                     | > CXXFLAGS; compilation options
|------------------------------------------------------------|
```

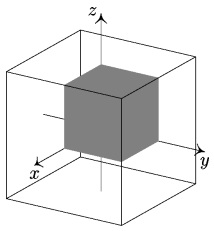
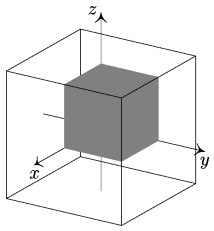
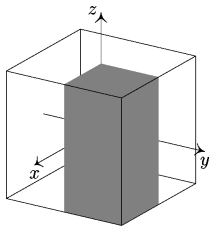
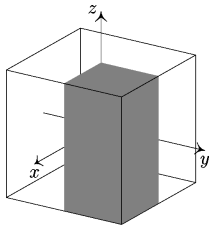
Tantalus.XXX.exe	source folder	mesh & spwfs	Typical use	Spatial representation
NLO.func make single	src	$(N_x/2, N_y/2, N_z/2, dx)$ (nwn/2, nwp/2)	Starting point	
NLO.func.T make single_T	src_T	$(N_x/2, N_y/2, N_z/2, dx)$ (nwn, nwp)	Nuclei with odd-N/Z Rotational bands	
NLO.func.P make single_P	src_P	$(N_x/2, N_y/2, N_z, dx)$ (nwn/2, nwp/2)	Octupole deformation Fission barriers	
NLO.func.TP make single_TP	src_TP	$(N_x/2, N_y/2, N_z, dx)$ (nwn, nwp)	Octupole for odd-Z/N Bands with octupole	

TABLE I. Summary of different source code sets currently available: default executable names, source code folders and (mesh,single-particle) number parameters are given with respect to a calculation with no symmetries imposed.

III. RUNNING TANTALUS

Any Tantalus executable can be used as

```
./Tantalus.[.....].exe < tant.in
```

This simple line masks however that there are multiple inputs and outputs.

```
Input
--------------------------------------------------------------------------------------------
(1) tant.in     :  REQUIRED
                   An input file detailing the specifics of the calculation, such as
                   nucleon number, name of the parameterization, mesh discretisation, ...
                   This is fed to the executable through STDIN.
(2) ${PARAM}.param file :  REQUIRED
                   An input file containing the requested the parameter values of the EDF.
                   Must be present in the current directory where Tantalus is executed.
                   Can be either a file named as
                    -  ${param}.param: named after the parameterization, ex. SLy4.param,
                                       containing a SINGLE parameter set.
                    -  forces.param  : a file containing multiple parameter sets.
                   Note, the code will look first for files names ${param}.param
                   and will only look for forces.param if the more specific file
                   cannot be found.
(3) ${INPUTFILENAME}.wf :  OPTIONAL
                   A wavefunction (.wf) file created by some previous Tantalus run
                   to warmstart the calculation. Must be present in the current directory
                   where Tantalus is executed.

Output
--------------------------------------------------------------------------------------------
(1) STDOUT       : ALWAYS PRODUCED
                   A wealth of information on all kinds of quantities, from the first
                   iteration to the last. If it is calculated by the code, then it is
                   printed here. All other outputs (with the exception of the single-particle
                   wavefunctions) are reformats for convenience of information that can in
                   principle be gotten from here. Can be captured through piping or the
                   'tee' command on Unix.
(2) ${OUTPUTFILENAME}.wf : ALWAYS PRODUCED
                   A .wf file containing the final state of the code on exit. Useful for
                   warmstarting the next calculations.
(3) ${BXLFIT}zXXXnYYY.out:  NOT PRODUCED BY DEFAULT
                   A one-line text file containing observables from the final iteration of
                   the code. Useful for automatic combining many calculations into, for
                   instance, a parameter fit. Name (and presence) determined by the
                   'BXLFIT' parameter discussed below.
(4) ${DENFILE} : NOT PRODUCED BY DEFAULT
                   File containing the neutron, proton and charge density at all the mesh
                   points.
(5) ${POTFILE}: NOT PRODUCED BY DEFAULT
                   File containing the neutron and proton potential associated with the
                   density, as well as Coulomb potential for both nucleon species at all
                   the mesh points.
(6) ${SPHFFILE} : NOT PRODUCED BY DEFAULT
                   File containing the properties of the single-particle wavefunctions in
                   the Hartree-Fock basis.
(7) ${SPCANFILE}: NOT PRODUCED BY DEFAULT
                   File containing the properties of the single-particle wavefunctions in
                   the canonical basis.
```

IV. INPUT

```
&nucleus         / ! Information on the nucleus and convergence
&mesh            / ! Mesh parameters
&func            / ! Name of the parameterization
&pairing         / ! Options linked to the treatment of pairing (and blocking!)
&indices         / ! Options and indices for blocking quasiparticles
                   ! NOTE: Optional
&evolution       / ! Options related to the evolution of the single-particle wavefunctions
&scfiteration    / ! Options for the SCF procedure: mixing of densities, potentials etc...
&wfs             / ! Number of wavefunctions and options for transformation
&IO              / ! Input and output options
                   ! (i)  multiple kinds of filenames
                   ! (ii) symmetry transformation options
&Inertia         / ! Determines for which l,m the collective inertias get calculated
&MomentParam     / ! General treatment of multipole constraints
&MomentConstraint/ ! Specify a single ( = one,1 ) multipole constraint.
                   ! NOTE: optional, can be repeated
&Cranking        / ! Information on cranking and determining angular momentum
```

FIG. 1. List of namelist inputs, in the correct order.

A. The STDIN input: tant.in

Tantalus reads input from STDIN, using the FORTRAN namelist machinery. The available namelists are indicated in Fig. 1; the ordering of the namelist is important and all of them are required. The only exceptions to this are (i) the &indices/ namelist, (ii) the &momentconstraint/ namelist and (iii) the &inertia/ namelist. All are only required if the user activates other parameters in the **&pairing/**, &MomentParam/ and &IO/ namelists. In addition, &momentconstraint/ can be repeated. Below, I document the input parameters in all of these namelists; please do not assume that all of the below is complete.

I have tried to indicate parameters that are either experimental and/or are unlikely to be useful for most people's use by "A" (for advanced) in the first column. I do not guarantee that these work correctly (although almost all do).

```
&nucleus
-------------------------------------------------------------------------------
    neutrons    = Number of neutrons. Can be non-integer.
    protons     = Number of protons. Can be non-integer.
    energy_prec = convergence criterion on the energy.
    moment_prec = convergence criterion on the multipole moments.
    disp_prec   = convergence criterion on the weighted dispersion of the spwfs.
    fermi_prec  = convergence criterion on the Fermi energy.
A   pairing_prec= precision required of the pairing solver.
                  [Recommendation: DO NOT TOUCH!]
A   inversetemp = inverse temperature, in units of MeV.
A   fixfermi    = Logical, if .true. keep the Fermi energies constant.
A   mun, mup    = Fermi energy for neutrons/protons, units of MeV.
-------------------------------------------------------------------------------

&mesh
-------------------------------------------------------------------------------
    nx/y/z      = number of points in x/y/z direction.
    dx          = mesh discretisation (in fm).
-------------------------------------------------------------------------------

&func
-------------------------------------------------------------------------------
    name_param  = name of the parametrization, needs to match a name on forces.param.
-------------------------------------------------------------------------------
```

```
&pairing
-------------------------------------------------------------------------------
    Type            = ‘HF’, ‘BCS’ or ‘HFB’.
    Guessgaps       = Logical. If .true. initialize a fixed starting guess for Delta’s.
    BlockType       = 0: No blocking
                      1: True blocking based on indices
                      2: True blocking based on lowest energy,
                      3: EFA based on indices
                      4: EFA based on lowest energy
    BlockNumber     = Number of quasiparticles to excite.
                      If Blocknumber > 0, read &Indices/ namelist after &pairing.
    pairingscheme   = 0 : direct solver   = fast but often unstable for odd-Z/N
                      1 : gradient solver = slow but almost guaranteed to converge
    Bogofromfile    = Logical. If .true. (= default value), use the Bogoliubov
                     transformation read from file to continue the calculation.
A   maxhfbiter      = Maximum number of iterations for the Fermisolver at each
                      mean-field iteration. (Default = 200)
A   FermiSolver     = Numerical solver to use to determine the Fermi energies
                      "Brent" => New and stable (default).
                      "Secant"=> Old and not so stable.
A   gradient_precon = Logical. If .true., use the gradient preconditioning scheme of
                     the Madrid group. Only active if pairingscheme = 1.
A   Particles_in_gas= Integer tracking the way the contribution from a free gas
                      is counted for the readjustment of the Fermi energy. Note
                      that this is only active at finite temperature and when
                      using BCS!
        0) No correction: the number of particles is the ordinary counting, Tr(rho).
        1) Subtraction  : the nucleus is modelled as being immersed in a gas of
                          free particles. The number of particles is Tr(rho) MINUS
                          the number of particles in a free gas at the same chemical
                          potential.
        2) Nogas        : the code is only allowed to have non-zero occupation
                          numbers for single-particle states with negative
                          single-particle energy.
A   Lambda_2        = Multiplier values for constraint on the dispersion of the
                      particle number. Default: (0.0,0.0).
-------------------------------------------------------------------------------

&Indices      (NOTE: this namelist is only read if BlockNumber >0à
-------------------------------------------------------------------------------
    BlockIndices = Blocknumber integers. These are the indices of the states in the
                   Hartree-Fock basis, for which the code will attempt to construct
                   quasiparticle excitations with large overlap with them.
    BlockLowest  = Set of ’a2’ formatted strings, of length BlockNumber.
                   ’n+’,’n-’ : lowest quasi-neutron of positive/negative parity
                   ’p+’,’p-’ : lowest quasi-proton of positive/negative parity
                   ’n0’,’p0’ : lowest quasi-neutron/proton of either parity.
                   Note that
                   (a) all of these can be combined. The input
                        ’n+’, ’n+’, ’p0’, ’p0’
                       will result in the code trying to construct the lowest state
                       with two-quasi-neutrons of positive parity and two
                       quasi-protons of whatever parity.
                   (b) If time-reversal is broken, this option will only consider
                       blocking options with positive signature. (Which makes no
                       difference if time-reversal is conserved)
-------------------------------------------------------------------------------
```

```
&evolution
-------------------------------------------------------------------------------
      maxiter     = maximum number of mean-field iterations.
      printiter   = Large printouts after every multiple of this value.
                    Attention: collective corrections are only recalculated at
                    every total printout, as determined by this parameter.
      gradient_safety   = Safety parameter for the estimation of the parameters
                          of the gradient scheme in units of MeV. Only active
                          when pairingscheme=1. Default: 0.1. Extra-safe value: 2.
A     Strategy          = Evolution strategy for the single-particle wavefunctions.
                           "HEAVYBALL" : heavy-ball scheme, default.
                           "IMTIME"    : imaginary time-evolution.
A     dt, momentum      = Parameters governing the heavy-ball update. Note: read
                          values will be overwritten if EstimateParams=.true.
A     Estimateparams    = If .true. (= default), use heuristics to guess optimal
                          evolution parameters dt and momentum at every iteration.
A     gradient_stepsize,= Parameters governing the heavy-ball update. Note: read
      gradient_mu         values will be overwritten if EstimateGradParams=.true.
                          Only active if pairingscheme=1.
A     EstimateGradparams= If .true. (= default), use heuristics to guess optimal
                          evolution parameters for the gradient solver in the
                          pairing space. Only active if pairingscheme=1.
-------------------------------------------------------------------------------

&scfiteration/
-------------------------------------------------------------------------------
A     scfscheme    = Integer determining the type of SCF evolution
                     0: preconditioning of F_I_I and F_I_S potentials
                     1: linear mixing of D_I_I and D_I_S densities
A     denmix       = Parameter governing the density mixing (Default: 0.75)
                        D_I_I_new = denmix * D_I_I_old + (1 - denmix) * D_I_I_new
A     preconfactor = Parameter governing the potential preconditioning
                        F_I_I_new = F_I_I_old + P^-1 (F_I_I_new - F_I_I_old)
                        P^-1 =  1 + preconfactor * Delta
-------------------------------------------------------------------------------

&wfs
-------------------------------------------------------------------------------
       nwn/nwp = number of neutron/proton wavefunctions.
                 Attention: these need to match the input .wf file, if present,
                 modulo the symmetry transformation rules discussed below.
A      osc_freq= Harmonic oscillator frequencies in all Cartesian directions
                 used for the generation of Nilsson orbitals if starting from
                 scratch. (omega_x, omega_y, omega_z) with typical size ~0.2 MeV.
                 Strictly spherical       :  omega_x = omega_y = omega_z
                 Prolate, symmetry axis Z :  omega_x = omega_y > omega_z
                 Oblate, symmetry axis Y :   omega_x = omega_z < omega_y
                 Default: (/ 0.2125, 0.2125, 0.175 /).
-------------------------------------------------------------------------------
```

```
&IO
-------------------------------------------------------------------------------
    Inputfilename  = name of the .wf file Tantalus will read. If this is 'INIT',
                     Tantalus will not read any file and initialize from scratch.
    Outputfilename = name of the .wf file Tantalus will write.
    BXLFIT         = string. Extra output will be written to the following file
                      '${BXLFIT}zXXXnYYY.out'
                     where XXX      = the number of protons  (forced to be integer)
                           YYY      = the number of neutrons (forced to be integer)
                           ${BXLFIT}= the value of the string.
                    The exact content of this file is explained below.
    denfile       =  filenames for the write-out of extra information. In order:
    potfile          scalar densities, mean-field potential, single-particle info
    sphffile         in the HF basis and single-particle info in the canonical basis.
    spcanfile        Finally, the inertfile outputs the (asked-for) collective
    inertfile        inertia parameters.
    N_inertia     =  number of collective degrees of freedom to calculate the
                     inertias for. If non-zero, the code will read the
                     &inertia/ namelist next.
    Extraspwfs    = 8 integers. If non-zero, the code will add this many (randomly
                    generated) spwfs in the corresponding isospin-parity-signature
                    blocks. Note that the nwn/nwp numbers need to be correctly
                    adapted, meaning nwn = nwn_file + number of new neutron spwfs.
    AllowTransform= Logical. If .true., it allows the code to transform the input,
                    meaning that you can either (a) add/remove mesh points
                                                (b) add extra spwfs (never remove!)
                                                (c) break a symmetry
A   checkpointiter= Write a checkpoint (wavefunction file) to disk every time this
                    many iterations have passed.
A   tofile        = filename for the writeout of time-odd densities D_I_S and
                    C_I_N to a file with this name.
A   blockfile     = filename to write the densities |psi|^2 to of the blocked
                    single-particle wavefunctions (in the canonical basis)
-------------------------------------------------------------------------------

&Inertia              * Optional, only read if N_inertia is > 0
-------------------------------------------------------------------------------
    Inertia_l     = lists of values l/m for collective degrees of freedom Qlm for
    Inertia_m       which the collective inertias should be calculated.
-------------------------------------------------------------------------------

&MomentParam
-------------------------------------------------------------------------------
      MoreConstraints = Logical. If .true., signals that the namelist
                        &MomentConstraint/ will follow.
      ContinueAll     = Logical. If .true., signal that the data of the constrained
                        multipole moments should be taken from the input .wf file, not from STDIN.
A     follow_com      = If .true., redefine the multipole moment operators at
                        every iteration to follow the centre-of-mass of the nucleus
                        around. Only active if parity is broken. Default: .false.
A     MaxMoment       = Maximum l of the multipole moments Q_lm to be calculated. Default = 10
A     MaxMomentMag    = Maximum l of the magnetic moments to be calculated.  Default = 3
A     radd, acut,       = Parameters of the cutoff function as pioneered by K. Rutz
      cutfac             for multipole constraints.  Defaults= (4 fm, 0.4fm, 10)
A     cutofftype      = Type of cutoff to use for the multipole constraints
                         0: Density-dependent cutoff of the K. Rutz-type
                         1: Spherical cutoff
-------------------------------------------------------------------------------
```

```
&MomentConstraint   ATTENTION: * only active if MoreConstraints = .true.
                               * this namelist can be repeated
-------------------------------------------------------------------------------
      l, m      = Integers, determining the multipole moment Qlm to constrain.
      Constraint      = Value to constrain <Qlm> to.
      Iteration       = If -1 (=default), Tantalus will constrain the moment for
                        the entire run. If other value, the constraint will only be
                        used for this number of iterations. Useful for breaking
                        self-consistent symmetries.
      MoreConstraints = If .true., signals that more &MomentConstraints namelists
                        will follow. If .false., signals that this is the last
                        occurrence of the &MomentConstraint namelist.
      Multfromfile    = Logical. If .true., start the calculation with the
                        Lagrange multiplier of the constraint as read from file.
A     iq1, iq2        = Real, alternative parameterisation of quadrupole moments.
                        Cannot be combined with values for "Constraint". Needs to
                        specify both a constraint on Q20 and Q22 to work.
A     Impart          = Logical. .false. if Real part, .true. if Imaginary part of the
                        multipole moment. No versions of the code support Impart=.true.
A     Intensity       = Real. Intensity of this constraint in the penalty function.
                        Default = 0.0. If left to default, the code will use a
                        heuristic to estimate a proper value.
A     Constrainttype  = Integer. How to handle this particular constraint:
                        0 : no constraint
                        1 : Augmented Lagrangian constraint (not recommended)
                        2 : Projection on feasible set constraint (Default)
A     Scalefactor     = Real. Rescaling factor for the projection step in the
                        case of constrainttype=2 constraints. Default=1.0
A     Intensityfactor = Real. Rescaling factor for the constraint intensity
                        when obtained from heuristic. Default = 1.0.
A     Isoswitch       = Integer.
                        0 : Default, constraint on the total density (protons+neutrons)
                        1 : Constraint on the neutron density
                        2 : Constraint on the proton density
-------------------------------------------------------------------------------

&Cranking
-------------------------------------------------------------------------------
      omegaX/Y/Z     = Real, cranking frequency (hbar omega_x/y/z) in the x/y/z
                       direction in  units of MeV. Non-zero values are not allowed
                       if time-reversal is not broken. Only omegaZ can take non-zero
                       values in all code versions discussed here.
      crankX/Y/Z     = Real, target value for the expectation value of the angular
                       momentum <J_x/y/z> around the relevant Cartesian axis.
      CrankTypeX/Y/Z = Integer, determines the type of cranking constraint.
                       0 : Default, cranking at constant omega.
                       1 : Cranking constraint through projection on feasible space
                           Update of omega to obtain targetted J_x/y/z at convergence.
      continuecrank  = Logical. If .true. use omegaX/Y/Z from the .wf file instead
                       of values read from STDIN.
A     IntensityX/Y/Z = Intensity of the cranking constraint for cranktype=1
                       calculations. Default = 0.0. If left to 0, the code will
                       use a heuristic to guess an appropriate value.
A     crank_smooth   = If .false. (=default), <J> is calculated for the readjustment
                       of the cranking constraints by summing the single-particle
                       expectation values of <J>, weighted with occupation factors.
                       If .true. <J> is instead calculated by integration of
                       current and spin densities.
-------------------------------------------------------------------------------
```

B. Output for the Brussels fitting codes

Output for the Brussels fitting codes is activated by entering a non-empty string for the `BXLFIT` parameter. The end-result is that Tantalus opens a file named `$BXLFITz$Zn$N.out` and writes a single line to it at the end of the run.

```
          N, Z, Total energy, Enocor, Erot+Evib,
     &    b20, b22, b30, b32, b40                                  &
     &    sqrt(<r^2_p>/Z), B(1:3), J2(1:3),                        &
     &    avgap_uv(n), avgap_uv(p), Epairn, Epairp                 &
     &    DEhistory, iter, iomsg
```

The individual entries are

```
 N,Z            = Average number of neutrons and protons                        [MeV]
Total Energy    = Total energy, including ALL terms.                            [MeV]
Enocorr         = Total mean-field energy, no collective corrections.           [MeV]
Erot + Evib     = Rotational and vibrational corrections.                       [MeV]
blm             = Deformation values \beta_{ell m} for the total density
sqrt(<r^2_p>/Z) = Root-mean square charge radius                                [fm]
B_x, B_y, B_z   = Collective Belyaev moments of inertia around Cartesian axes   [MeV^-1 hbar^2]
J2_x, J2_y, J2_z= Expecation value of <J^2> around Cartesian axes               [hbar^2]
avgap_uv(n/p)   = is the average gap for protons or neutrons as calculated for  [MeV]
                  the selected pairing approximation (HF/BCS/HFB) by either
                  averaging the gaps with the anomalous density kappa (uv) for
                  protons and neutrons.
Epairn,Epairp   = Pairing energy of neutrons and protons                        [MeV]
iter            = Number of SCF iterations performed by the code before exiting.
iomsg           = Output message by the code. Currently can be
        CONVERGED : the code exited because it judged convergence to be reached.
        MAXITER   : the code exited because it reached its maximum of iterations.
        CHECKPOINT: this wavefunction file was written as a checkpoint
```

C. The structure of a parameterization (.param) file

The parameterization file $name_param.param contains all the values and special flags that characterize a parameterization. The structure of the .param file needs to match the executable, as not every form depends on the same parameters. This means that there are multiple type of .param files, although only one is available in the repository right now. It concerns:

- NLO EDF in the style of the Brussels group. Form as in G. Scamps et al., Eur. Phys. J. A 57, 333 (2021).
- ...

These are documented separately below. Note that, in all cases, the parameters that figure in the definition of the EDF are **REQUIRED** to be read from file. These parameters are indicated below with (R), all others can be omitted, in which case they will take (hopefully sensible) default values.

1. NLO EDF in the style of the Brussels group

```
&skf
name     = "SLy4"         (R) ! Name of the parameterization.
                              ! This needs to match name_param in the input data
func_file= 'NLO.func'     (R) ! Name of the functional file used for compilation.
t0       =  -2488.913,    (R) ! t0 parameter of the Skyrme parameterization
t1       =    486.818,    (R)
t2       =   -546.395,    (R)
t3       =  13777.0,      (R)
t4       =      0.0       (R)
t5       =      0.0       (R)
x0       =      0.834,    (R)
x1       =     -0.344,    (R)
x2       =     -1.0,      (R)
x3       =      1.354,    (R)
x4       =      0.0       (R)
x5       =      0.0       (R)
beta     =      0.0       (R) ! Exponent of the density dependence (t4).
gamma    =      0.0       (R) ! Exponent of the density dependence (t5).
sigma    =      0.1666667 (R) ! Exponent of the density dependence (t3).
wso      =    123.0,      (R) ! Spin-orbit strength.
wsoq     =    123.0       (R) !
CT0      =      0.0       (R) ! Coupling constant (isoscalar) of the J^2 terms
CT1      =      0.0       (R) ! Coupling constant (isovector) of the J^2 terms
CSDS0    =      0.0       (R) ! Coupling constant (isoscalar) of the s Delta s term
CSDS1    =      0.0       (R) ! Coupling constant (isovector) of the s Delta s term
Vn       =  -1250.00000   (R) ! Pairing strength (neutrons) of the ULB interaction.
Vp       =  -1250.00000   (R) ! Pairing strength (protons) of the ULB interaction.
alpha    =    1.0         (R) ! Alpha parameter in the ULB pairing interaction.
sigma_p  =    1.0         (R) ! Power of the density dependence in the pairing interaction.
                              ! Alpha in EQ. (2) in S. Goriely et al, PRC88, 061302 (2013).
Estabp   =    0.0             ! Cutoff factors for the stabilized pairing
Estabn   =    0.0             ! functional of J. Erler et al., EPJA 37 (2008).
                              ! Set to zero to disable.
                              ! Typical value when active = 0.3 MeV
hbm(1)   = 20.73551910,
           20.73551910        ! Value of hbar^2/2m
force_switch = 0.0            ! INACTIVE
COM1Body = 2                  ! Treatment of the one-body COM correction.
                              !   (0) => None. (1) => Perturbative. (2) => Self-consistent
COM2Body = 0                  ! Treatment of the two-body COM correction.
                              !   (0) => None. (1) => Perturbative.
CoulTreatment = 1             ! Treatment of Coulomb interaction.
                              !   (0) => No Coulomb included.
                              !   (1) => Direct and exchange (Slater) contribution
                              !   (2) => Only direct contribution.
protonsize  =  0.0, 0.0       ! Rms radii  (r+, r-) of a double Gaussian model to be used for the proton.
                              ! If 0.0, no folding is used for either Gaussian.
neutronsize =  0.0, 0.0       ! Rms radii  (r+, r-) of a double Gaussian model to be used for the neutron.
                              ! If 0.0, no folding is used for either Gaussian.
                              ! Note that input for protons and neutrons is NOT analogous.
                              ! The input for protons is the proton rms-radius, while for neutrons is
                              ! r_+^2 , r_-^2 in the model of Chandra & Sauer PRC13, 245
HOCOMform   = .false.         ! Correct the above rms radii for the c.m. correction in a HO basis?
nucleonsize_selfconsistent    ! Whether to use the folding self-consistently,
            = .true./         ! or only in the calculation of the energy.
rotcorr     = 0               ! Whether (1) or not (0) to include the rotational
                              ! correction and vibrational correction (!) in
                              ! calculation.
rotcorrb    = 0               ! Parameters b and c for the contribution of the
rotcorrc    = 0               ! rotational correction.
vibcorrb    = 0               ! Parameters b, d and l for the vibrational
vibcorrd    = 0               ! correction.
vibcorrl    = 0               !
eps         = 1d-20           ! Numerical parameter to safeguard the potentials
                              ! associated with density-dependent terms with
                              ! small exponents. Default value works for everything
                              ! without t4/t5 terms in my experience.
```

V. APPLYING TRANSFORMATIONS

The code is capable of modifying a lot of aspects of information read from a .wf file. This section details a little bit of what is possible at the moment:

1. Modify the number of mesh points
2. Add single-particle wavefunctions
3. Break **a single** self-consistent symmetry

Important caveats are

- **Only one of these actions is allowed in any given run**. The code is hard enough to keep straight in that way, so I do not (and will not) support combinations in one go. If you want to do multiple things (say, break parity and add points along the z-axis), simply do them in successive runs.
- The code will only allow you to do any of these if AllowTransform = .true. in the &IO/ namelist. Activating this trigger will remove a bunch of sanity checks on the input though, and it will be **very easy** to make mistakes.

Since it is very easy to do things wrongly, the code will always try to do its best to tell you what it actually did in the header, such as for example the following:

```
-------------------------------------------------------
| Transformation of the input                         |
| On file:                                            |
|    nx, ny, nz    =    12   12   24                  |
|    dx            =  0.8000000 (fm)                  |
|    nwn, nwn      =      20     20                   |
|    (n+,n-)       = (   20    0    0    0)          |
|    (p+,p-)       = (   20    0    0    0)          |
| This calculation:                                   |
|    nx, ny, nz    =    12   12   28                  |
|    dx            =  0.8000000(fm)                   |
|    nwn, nwn      =      20     20                   |
|    (n+,n-)       = (   20    0    0    0)          |
|    (p+,p-)       = (   20    0    0    0)          |
|_____________________________________________________|
```

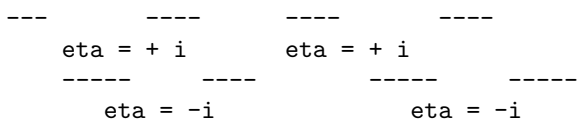
A. Mesh modifications

Mesh modification is as simple as specifying the new mesh parameters in the input and setting AllowTransform = .true.. The code will transform the single-particle wavefunctions on the mesh by either cutting or adding points. Values of single-particle wavefunctions on any new point are initially set to zero. Note that the **user is responsible** for verifying that the box remains big enough if removing points; the code will do what you ask and not verify anything. Changing dx is perfectly possible, but does in my experience not gain anything in practice.

B. Wavefunction modifications

Number of wavefunction modification is slightly more subtle: one should set AllowTransform = .true. and use the extraspwfs flag. This flag contains the extra wavefunctions as ordered in the symmetry-blocks of isospin/parity/signature.

```
extraspwfs = n++, n+-, n-+, n--, p++, p+-, p-+, p--
             ------------------  -------------------
               neutrons             protons
             ---------  --------  --------  --------
               P = +1    P = -1    P = +1    P = -1
```



What these numbers should be depend on the symmetries of the calculation.

- Time-reversal conserved: all $\eta = -1$ numbers should be zero.
- Time-reversal broken: all $\eta = -1$ numbers should be equal to the $\eta = +1$ counterparts.
- Parity broken: all $P = -1$ values should be zero.

For example, extraspwfs = 10,0,6,0,10,0,6,0 in a default calculation would add 10 positive parity single-particle wavefunctions for each nucleon species and 6 negative parity single-particle wavefunctions.
Note that in all cases the new total number of wavefunctions should be consistent with what was read from file and what you have hadded, meaning that

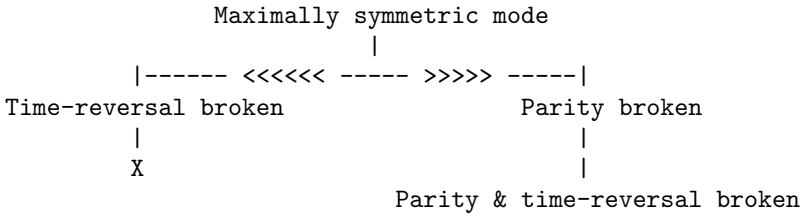
$$\mathsf{nwn} = \mathsf{nwn}_{\mathrm{file}} + \sum_{i=1}^{4} \mathsf{extraspwfs[i]}\,, \tag{1}$$

$$\mathsf{nwp} = \mathsf{nwp}_{\mathrm{file}} + \sum_{i=5}^{8} \mathsf{extraspwfs[i]}\,. \tag{2}$$

All new single-particle wavefunctions are initialized randomly an set at very high energy at the start of iterations.

C. Symmetry transformations

Wavefunction files from calculations with **more imposed symmetries** can be used to warmstart calculations with **less imposed symmetries**. Practical reasons limit any executabler to (1) breaking one symmetry in any given run and (2) to be capable of breaking only one single breaking symmetry. The practical consequences of this are illustrated as follows:



I.e. from maximally symmetric one can go in the direction of parity breaking or time-reversal breaking. From the parity-broken mode, one can go to the parity and time-reversal broken mode. The transformation from Time-reversal broken to PT-broken is NOT SUPPORTED in this repository, although it could be easily provided if TRULY needed. To perform a symmetry transformation, simply give a symmetry-conserving .wf file to a symmetry-breaking executable, with the flag AllowTransform=.true.. You only need to make sure that the new degrees of freedom match the symmetry-broken version of the input file. Meaning that

- When breaking time-reversal, the total number of wavefunctions should be doubled.

$$\mathsf{nwn} = 2 \times \mathsf{nwn}_{\mathrm{file}}$$
$$\mathsf{nwp} = 2 \times \mathsf{nwp}_{\mathrm{file}}.$$

- When breaking parity, the total number of points in the z-direction should be doubled

$$\mathsf{nz} = 2 \times \mathsf{nz}_{\mathrm{file}}$$